DAI Sports Slice Architecture Study Guide

Current Sports Matchup Analysis Flow

purpose:

- give a study-first view of the current sports slice
- keep attention on the load bearing request flow and seams
- support shipping the next slice without carrying minor complexity in working memory

current boundary:

- reflects the sports matchup analysis flow as currently implemented
- focuses on the sports slice and the shared platform path it exercises

primary docs used first:

- `dai-vault/02 Platform/architecture/current-sports-analysis-flow.md`
- `dai-vault/02 Platform/architecture/orchestration.md`
- `dai-vault/02 Platform/architecture/current-agent-run-contract.md`
- `dai-vault/06 Execution/handoffs/current-sports-matchup-analyzer.md`

Page 0

Prepared as a print study handout for the current sports matchup analysis path.

Page 0

Executive Map

the current sports slice has two real paths.

the first path is the reference path:

- angular asks `.NET` for `GET /api/competitions`
- angular asks `.NET` for `GET /api/competitions/{code}/teams`
- angular asks `.NET` for `GET /api/competitions/{code}/matchup-dates?teamA=...&teamB=...`
- `.NET` resolves the real scheduled event and returns `homeTeam` and `awayTeam`

that path matters because the analyze call depends on a real event, not a guessed orientation.

the second path is the analysis path:

- angular submits `CreateAgentRunRequest` to `POST /api/agent-runs`
- `.NET` resolves trust through `IdentityResolver`
- `.NET` creates a pending `AgentRun`
- `AgentRunService` runs `collect -> analyze -> evaluate -> compose`
- collector clients retrieve grounded evidence
- `FastApiClient` sends the typed request to FastAPI at `POST /api/sports/analyze`
- FastAPI validates competition and dispatches to the correct analyzer
- FastAPI hands the prompt to OpenAI through the python client
- `.NET` calibrates final confidence and stores the completed run
- angular receives `AgentRunResultDto` and renders the read

the main architectural point:

- angular owns interaction
- `.NET` owns trust, persistence, collection, calibration, and composition
- python owns synthesis

the slice is no longer "frontend calls model and gets a summary." it is now a platform-mediated request with explicit evidence, evaluation, and durable run records.

Happy Path

1. `dai/apps/sports-app/src/app/analyzer/analyzer.component.html` class: `AnalyzerComponent` template method: `(ngSubmit)="analyze()"` responsibility: submits the configured matchup from the angular form.

2. `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts` class: `AnalyzerComponent` method: `analyze()` responsibility: builds `CreateAgentRunRequest` from `selectedEvent()` so the request uses resolved `homeTeam` / `awayTeam`.

3. `dai/apps/sports-app/src/app/sports-api.service.ts` class: `SportsApiService` method: `createMatchupAnalysis(req)` responsibility: posts the typed analysis request to `.NET` at `POST /api/agent-runs`.

4. `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs` class: `AgentRunsController` method: `Create(req, ct)` responsibility: resolves identity, creates a pending `AgentRun`, and delegates execution to the application service.

5. `dai/platform/dotnet/DevCore.Api/Identity/IdentityResolver.cs` class: `IdentityResolver` method: `ResolveAsync(ct)` responsibility: maps the current request to tenant and user keys and establishes the trust boundary.

6. `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs` class: `AgentRunService` method: `ExecuteAsync(req, agentRunId, ct)` responsibility: dispatches by `RunType` to the sports matchup analysis path.

7. `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs` class: `AgentRunService` method: `ExecuteSportsMatchupAsync(req, agentRunId, ct)` responsibility: runs the core `collect -> analyze -> evaluate -> compose` sequence.

8. `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs` class: `AgentRunService` method: `CollectAsync(input, gameDate, ct)` responsibility: retrieves competition-specific grounded evidence before the model call.

9. `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs` and `dai/services/agent-service/app/routes/sports.py` class: `FastApiClient` and FastAPI route module method: `AnalyzeSportsMatchupAsync(req, agentRunId, ct)` and `analyze_sports(req, request)` responsibility: send the typed request to FastAPI at `POST /api/sports/analyze`, include correlation headers, validate competition, and dispatch to the correct analyzer family.

10. `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs` and `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts` class: `AgentRunService` and `AnalyzerComponent` method: `Evaluate(...)`, `ComposeDecisionArtifact(...)`, and the success callback in `analyze()` responsibility: calibrate final confidence, persist the completed run, return `AgentRunResultDto`, and update angular state for rendering.

API Flow at a Glance

1. Angular to Reference API

caller:

- `dai/apps/sports-app/src/app/sports-api.service.ts`

endpoints:

- `GET /api/competitions`
- `GET /api/competitions/{code}/teams`
- `GET /api/competitions/{code}/matchup-dates?teamA=...&teamB=...`

purpose:

- build the selection state
- resolve a real scheduled event
- produce `selectedEvent()` for the analysis payload

important point:

- this path stays entirely inside angular + `.NET`
- python is not involved

2. Angular to `.NET` Analysis API

caller:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
- `dai/apps/sports-app/src/app/sports-api.service.ts`

endpoint:

- `POST /api/agent-runs`

request body shape:

```
{
  "runType": "sports.matchup.analysis",
  "agentProfileId": null,
  "input": {
    "competition": "nba",
    "homeTeam": "Boston Celtics",
    "awayTeam": "New York Knicks",
    "gameDate": "2026-04-22"
  }
}
```

purpose:

- enter the trusted platform path
- create a durable run
- execute the analysis pipeline

3. `.NET` to FastAPI Analysis API

caller:

- `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs`

endpoint:

- `POST /api/sports/analyze`

headers introduced here:

- `X-Correlation-Id`
- `X-Agent-Run-Id`

request body shape:

```
{
  "competition": "nba",
  "homeTeam": "Boston Celtics",
  "awayTeam": "New York Knicks",
  "gameDate": "2026-04-22",
  "basketballScheduleContext": {
    "homeLastGameDate": "2026-04-20",
    "homeDaysRest": 1,
    "homeIsBackToBack": false,
    "awayLastGameDate": "2026-04-21",
    "awayDaysRest": 0,
    "awayIsBackToBack": true
  },
  "basketballMarketContext": {
    "homeTeam": "Boston Celtics",
    "awayTeam": "New York Knicks",
    "homeSpread": "-4.5",
    "awaySpread": "+4.5",
    "bookmaker": "draftkings",
    "updatedAt": "2026-04-22T14:05:00Z"
  }
}
```

important point:

- `.NET` sends only the grounded evidence it actually retrieved
- missing evidence is represented as `null`, not guessed context

4. FastAPI to OpenAI

caller:

- `dai/services/agent-service/app/services/sports_analyzer.py`

call:

- `client.chat.completions.create(...)`

purpose:

- take the typed request plus grounded context blocks
- synthesize `lean`, `summary`, `confidence`, and `factors`

important point:

- this is an internal service-to-model call, not another app-facing API contract

5. Response Back to Angular

path:

- FastAPI returns `SportsAnalysisResponse`
- `.NET` evaluates and composes
- `.NET` returns `AgentRunResultDto`
- angular success callback stores `result()` and `analyzedMatchup()`

response body shape returned to angular:

```
{
  "agentRunId": "guid",
  "status": "completed",
  "lean": "edge toward Celtics based on rest advantage.",
  "summary": "...",
  "confidence": 0.67,
  "factors": ["market: celtics favored by 4.5", "rest/fatigue: knicks on back-to-back"],
  "createdUtc": "2026-04-22T14:05:10Z",
  "durationMs": 1842
}
```

External Sources in the Current Flow

What Is Actually Called Today

`dai/platform/dotnet/DevCore.Api/Program.cs` wires four external-facing clients into the current sports slice.

The Odds API

files:

- `dai/platform/dotnet/DevCore.Api/Sports/OddsScheduleClient.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/OddsMarketClient.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/CompetitionCatalog.cs`

base url:

- `https://api.the-odds-api.com`

endpoints used:

- `GET /v4/sports/{oddsApiKey}/events`
- `GET /v4/sports/{oddsApiKey}/odds?regions=us&markets=spreads...`

what it provides:

- matchup event lookup for the reference path
- real home/away orientation for the selected event
- current spread context for football and basketball enrichment

important point:

- the sport-specific `oddsApiKey` comes from `CompetitionCatalog`
- the api key lives in `.NET` config as `OddsApi:ApiKey`
- failures degrade to empty schedule results or `null` market context rather than failing the whole run by default

ESPN Scoreboard API

file:

- `dai/platform/dotnet/DevCore.Api/Sports/EspnBasketballScheduleClient.cs`

base url:

- `https://site.api.espn.com`

endpoints used:

- `GET /apis/site/v2/sports/basketball/nba/scoreboard?dates={yyyyMMdd}`
- `GET /apis/site/v2/sports/basketball/mens-college-basketball/scoreboard?dates={yyyyMMdd}&groups=50&limit=500`

what it provides:

- each team's most recent game date
- derived `daysRest` and `isBackToBack` context for `nba` and `ncaamb`

important point:

- `.NET` turns raw scoreboard payloads into a thin typed `BasketballScheduleContext` before FastAPI sees them
- FastAPI never calls ESPN directly in this slice

MLB Stats API

file:

- `dai/platform/dotnet/DevCore.Api/Sports/MlbStarterClient.cs`

base url:

- `https://statsapi.mlb.com`

endpoint used:

- `GET /api/v1/schedule?sportId=1&date={gameDate}&hydrate=probablePitcher`

what it provides:

- probable starter names
- probable starter handedness

important point:

- this is public and does not require an api key
- partial data is rejected; if both starters are not available, the collector returns `null`

OpenAI

file:

- `dai/services/agent-service/app/services/sports_analyzer.py`

call:

- `AsyncOpenAI(...).chat.completions.create(...)`

what it provides:

- synthesis only: `lean`, `summary`, provisional `confidence`, and `factors`

important point:

- OpenAI is not an evidence source in this architecture
- FastAPI does not call Odds API, ESPN, or MLB Stats API directly
- the model only sees what `.NET` already grounded and serialized into `SportsAnalysisRequest`
- the key for this boundary lives in FastAPI environment configuration as `OPENAI_API_KEY`

External Call Ownership

- angular never calls third-party providers directly
- `.NET` owns external evidence retrieval, provider-specific mapping, caching, and graceful fallback
- FastAPI owns typed parsing and synthesis, not evidence collection
- in the scoped files reviewed, deterministic external grounding exists for schedule orientation, market spreads, basketball rest, and mlb probable starters
- in the scoped files reviewed, there is no dedicated deterministic collector yet for categories like injury reports or weather

Layer Responsibilities

Angular UI

files:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
- `dai/apps/sports-app/src/app/analyzer/analyzer.component.html`
- `dai/apps/sports-app/src/app/sports-api.service.ts`

owns:

- user interaction
- sport / level / team / date selection
- request submission
- result state and rendering

does not own:

- trust

- evidence retrieval
- final confidence
- persistence

Reference API Path

files:

- `dai/platform/dotnet/DevCore.Api/Controllers/SportsReferenceController.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/OddsScheduleClient.cs`

owns:

- competition lookup
- team lookup
- matchup date lookup
- real event orientation

why it exists:

- the analysis call needs a real `homeTeam` / `awayTeam`, not a user guess

AgentRunsController

file:

- `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`

owns:

- HTTP entrypoint
- identity handoff
- pending/completed/failed run persistence
- returning the trimmed UI DTO

why it exists:

- keeps HTTP and persistence concerns out of orchestration

IdentityResolver

files:

- `dai/platform/dotnet/DevCore.Api/Identity/IIdentityResolver.cs`
- `dai/platform/dotnet/DevCore.Api/Identity/IdentityResolver.cs`

owns:

- tenant and user resolution
- authenticated claim mapping
- development bypass behavior

why it exists:

- this is the trust boundary

IAgentRunService / AgentRunService

files:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/IAgentRunService.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

owns:

- run-type dispatch
- sports orchestration
- collect / analyze / evaluate / compose flow

why it exists:

- this is the application seam between controller and execution logic

Collector Clients

files:

- `dai/platform/dotnet/DevCore.Api/Sports/OddsScheduleClient.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/OddsMarketClient.cs`
- `dai/platform/dotnet/DevCore.Api/Sports/EspnBasketballScheduleClient.cs`

- `dai/platform/dotnet/DevCore.Api/Sports/MlbStarterClient.cs`

owns:

- external evidence retrieval
- deterministic, typed platform inputs

why they exist:

- grounded data should enter before the model call, not be guessed inside it

current providers:

- The Odds API for event lookup and spread enrichment
- ESPN scoreboard data for basketball rest grounding
- MLB Stats API for probable starters
- OpenAI is downstream synthesis, not a collector

FastApiClient

file:

- `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs`

owns:

- typed handoff from `.NET` to FastAPI
- correlation headers
- transport errors

why it exists:

- it keeps HTTP integration out of orchestration logic

FastAPI Route / Analyzer

files:

- `dai/services/agent-service/app/routes/sports.py`
- `dai/services/agent-service/app/services/sports_analyzer.py`

owns:

- competition validation on the python side
- analyzer-family dispatch
- pydantic request parsing and response shaping
- model prompt construction
- `lean`, `summary`, `confidence`, and `factors`

does not own:

- trust
- provenance
- final confidence
- persistence

how the Pydantic models work here:

- `SportsAnalysisRequest` is a `BaseModel` that parses the incoming JSON body into typed python objects before the route logic uses it
- nested context objects like `FootballMarketContext` and `BasketballScheduleContext` are also `BaseModel` types, so FastAPI hands the route a structured request object instead of raw dictionaries
- optional fields such as `basketballScheduleContext: ... | None = None` mean the field may be absent or `null` when `.NET` could not ground that evidence
- `response_model=SportsAnalysisResponse` tells FastAPI to shape the route output to the expected response contract

practical meaning:

- pydantic is doing boundary validation and typed parsing
- it is not deciding business logic
- it is not deciding confidence
- it is the typed contract layer on the python side

Evaluator / Compose Seam

files:

Page 0

- `dai/platform/dotnet/DevCore.Api/AgentRuns/EvaluatorOutput.cs`

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

owns:

- calibrated final confidence
- keeping analyzer confidence separate
- turning collector + analyzer + evaluator output into one stored result

why it exists:

- the platform needs to distinguish narrative confidence from evidence-backed confidence

Persistence

files:

- `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunContracts.cs`
- `dai-vault/02 Platform/architecture/current-agent-run-contract.md`

owns:

- pending/completed/failed run state
- input/output audit trail
- run duration and correlation ids

why it exists:

- this slice needs a durable record of what happened, not just a transient UI response

Interface and Seam Inventory

`IIdentityResolver`

file:

- `dai/platform/dotnet/DevCore.Api/Identity/IIdentityResolver.cs`

why it matters:

- separates controller logic from trust resolution

- marks the tenant/user boundary clearly

`IAgentRunService`

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/IAgentRunService.cs`

why it matters:

- keeps the controller independent from sports-specific execution
- gives a clean insertion point for more run types later

`RunTypes` Dispatch

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunContracts.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

why it matters:

- one place decides which execution path handles a run

`SportsCollectorOutput`

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/SportsCollectorOutput.cs`

why it matters:

- this is the single source of truth for grounded evidence and `GroundedSignals`

`EvaluatorOutput`

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/EvaluatorOutput.cs`

why it matters:

- keeps final confidence as a platform concern, not a raw model field

`ComposeDecisionArtifact(...)`

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

why it matters:

- it is the seam where analyzer language, collector provenance, and evaluator confidence become one result

FastAPI Contract Boundary

files:

- `dai/platform/dotnet/DevCore.AiClient/SportAnalysisContracts.cs`
- `dai/services/agent-service/app/models/sports.py`

why it matters:

- this is the typed `.NET` / python boundary for the analysis call

`CompetitionCatalog`

file:

- `dai/platform/dotnet/DevCore.Api/Sports/CompetitionCatalog.cs`

why it matters:

- centralizes competition identity, display labels, provider keys, and evidence richness expectations

Contracts That Matter

`CreateAgentRunRequest`

file:

- `dai/apps/sports-app/src/app/core/models/agent-run.model.ts`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunContracts.cs`

what it does:

- carries `runType` plus the sports analysis input from angular to `.NET`

why it matters:

- this is the top-level entry contract for the analysis path

what breaks if it drifts:

- controller binding
- run dispatch
- input persistence
- frontend submit behavior

`MatchupEventDto`

file:

- `dai/apps/sports-app/src/app/core/models/agent-run.model.ts`
- `dai/platform/dotnet/DevCore.Api/Sports/OddsScheduleClient.cs`

what it does:

- represents the real scheduled event with resolved `homeTeam` and `awayTeam`

why it matters:

- it converts neutral team selection into the real event used for analysis

what breaks if it drifts:

- wrong orientation in the analysis payload

- bad market matching
- incorrect collector inputs

`SportsAnalysisRequest`

file:

- `dai/platform/dotnet/DevCore.AiClient/SportAnalysisContracts.cs`
- `dai/services/agent-service/app/models/sports.py`

what it does:

- carries competition, matchup, date, and grounded evidence from `.NET` to FastAPI

why it matters:

- this is the core cross-runtime handoff

api detail:

- `.NET` serializes the request with camelCase json keys
- the python `BaseModel` fields use those same camelCase names
- nested context objects are parsed into typed model instances before `analyze_sports(...)` runs

what breaks if it drifts:

- FastAPI dispatch
- prompt inputs
- collector-to-analyzer wiring

`SportsAnalysisResponse`

file:

- `dai/platform/dotnet/DevCore.AiClient/SportAnalysisContracts.cs`
- `dai/services/agent-service/app/models/sports.py`

what it does:

- returns analyzer output: `lean`, `summary`, `confidence`, `factors`

why it matters:

- evaluation and final composition depend on it

api detail:

- FastAPI returns a plain route object matching `SportsAnalysisResponse`
- Pydantic shapes that output to the expected response schema
- `.NET` then deserializes it and treats `confidence` as provisional, not final

what breaks if it drifts:

- `.NET` evaluation
- result composition
- final UI payload

`SportsCollectorOutput`

file:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/SportsCollectorOutput.cs`

what it does:

- holds grounded evidence and computes `GroundedSignals`

why it matters:

- provenance and evidence richness depend on it

what breaks if it drifts:

- confidence calibration logic
- stored evidence quality
- future learning-loop reliability

Evidence Grounded Analysis Path

Collect

happens in:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`
- collector clients under `dai/platform/dotnet/DevCore.Api/Sports/`

what it adds:

- real external evidence before the model call
- explicit `null` when evidence is missing
- grounded signal ownership in `.NET`

simple rule:

- the collector decides what was real for this run

Analyze

happens in:

- `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs`
- `dai/services/agent-service/app/routes/sports.py`
- `dai/services/agent-service/app/services/sports_analyzer.py`

what it adds:

- family-aware synthesis
- `lean`, `summary`, `factors`
- provisional analyzer confidence

api detail:

- `.NET` calls FastAPI through `FastApiClient.AnalyzeSportsMatchupAsync(...)`
- FastAPI route `analyze_sports(...)` receives a typed `SportsAnalysisRequest`
- the route reads `X-Correlation-Id` and `X-Agent-Run-Id` from headers for logging and traceability
- the route dispatches by competition family to `analyze_football(...)`, `analyze_basketball(...)`, or `analyze_mlb(...)`
- those analyzer functions build prompt blocks only from grounded context that is present

simple rule:

- python writes the read from the evidence it is given

Evaluate

happens in:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`
- `dai/platform/dotnet/DevCore.Api/AgentRuns/EvaluatorOutput.cs`

what it adds:

- calibrated final confidence
- preserved analyzer confidence
- confidence bands driven by evidence richness

simple rule:

- the model can sound confident without real evidence
- the evaluator corrects for that

Compose

happens in:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`

what it adds:

- one application result combining analyzer language, collector provenance, and evaluator confidence

simple rule:

- this is where the result becomes the platform's output, not just the model's output

Evidence Richness and Confidence

current behavior:

- zero grounded signals: strong dampening
- partial grounding: lighter dampening
- full currently supported grounding: least restrictive current path

important point:

- displayed confidence is not the raw model number
- displayed confidence is the evaluator's calibrated value

Failure Path

Invalid Selection or Missing `selectedEvent`

owner:

- angular

files:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`

surface:

- `analyze()` returns early if the form is invalid or `selectedEvent()` is missing
- no backend call is made

Reference Lookup Failure

owner:

- angular for UI handling
- `.NET` reference controller for validation responses

files:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
- `dai/platform/dotnet/DevCore.Api/Controllers/SportsReferenceController.cs`

surface:

- loading teams or dates fails
- angular sets `error`
- the user remains in configuration state

Identity Resolution Failure

owner:

- `.NET`

files:

- `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`
- `dai/platform/dotnet/DevCore.Api/Identity/IdentityResolver.cs`

surface:

- controller returns `401`
- angular lands in the error callback

Collector Failure

owner:

- `.NET`

files:

- `dai/platform/dotnet/DevCore.Api/AgentRuns/AgentRunService.cs`
- collector client files under `dai/platform/dotnet/DevCore.Api/Sports/`

surface:

- expected path is graceful degradation to `null` evidence when the client can fail closed
- request can still succeed with weaker grounding and lower confidence
- if a client throws instead, the whole execution fails

FastAPI Failure

owner:

- FastAPI for route/model errors
- `.NET` for surfacing and run state

files:

- `dai/platform/dotnet/DevCore.AiClient/FastApiClient.cs`
- `dai/services/agent-service/app/routes/sports.py`

surface:

- unsupported competition or runtime/model error becomes HTTP failure
- `.NET` logs and throws
- the run is marked failed if persistence reaches the catch path

inference from FastAPI + Pydantic usage:

- if the incoming JSON body did not match the `SportsAnalysisRequest` model, FastAPI would reject it before `analyze_sports(...)` ran
- this is framework behavior from the `BaseModel` route binding, not custom error handling in this repo

Persistence or Execution Failure

owner:

- `.NET`

files:

- `dai/platform/dotnet/DevCore.Api/Controllers/AgentRunsController.cs`

surface:

- initial save can fail before the run exists
- execution or final save can fail after the pending row exists
- controller attempts to preserve failure state on the run

Angular Error Rendering

owner:

- angular

files:

- `dai/apps/sports-app/src/app/analyzer/analyzer.component.ts`
- `dai/apps/sports-app/src/app/analyzer/analyzer.component.html`

surface:

- error callback sets `error`

- template shows the generic failure copy in the result rail and narration strip

Complexity Watchlist

`AgentRunService`

why acceptable now:

- it is the right home for current sports orchestration

warning sign:

- too many competition branches or repeated source-specific conditionals in `CollectAsync()`

`AnalyzerComponent`

why acceptable now:

- one component still owns one workflow

warning sign:

- more analysis modes, more response variants, or more backend coordination logic land in the same class

TS / C# / Python Contract Duplication

why acceptable now:

- the number of contracts is still small and explicit

warning sign:

- routine field changes create drift, repeated bugs, or cross-runtime mismatches

`OutputJson` as Blob Storage

why acceptable now:

- the artifact is still evolving and shipping speed matters more than queryability

warning sign:

- you need structured analytics, stable downstream processing, or richer history surfaces

Competition-Specific Branching in the Platform Layer

why acceptable now:

- docs explicitly treat this as the correct deferral before a profile/config system is justified

warning sign:

- every new competition requires scattered manual changes across many files with no central pattern

What I Need In My Head Before Shipping the Next Slice

- the analyze request is only safe because angular uses a real `selectedEvent()`
- `AgentRunsController.Create()` is the HTTP, trust, and persistence entrypoint
- `IdentityResolver.ResolveAsync()` is where tenant-scoped trust begins
- `AgentRunService` is the load bearing execution center of this slice
- the core path is `collect -> analyze -> evaluate -> compose`
- grounded evidence belongs to `.NET`, not the model
- the current grounded external sources are The Odds API, ESPN scoreboard data, and MLB Stats API
- final confidence belongs to the evaluator, not the model
- `CompetitionCatalog` is the first place to look for competition behavior
- `SportsCollectorOutput` is the provenance seam worth protecting

Active Recall Page

Flashcard Prompts

1. where does a sports matchup request first become tenant-scoped?
2. why does angular use `selectedEvent()` instead of Team A / Team B order for the analysis payload?
3. what is the difference between the reference path and the analysis path?
4. what does `IAgentRunService` protect the controller from knowing?
5. what is the single source of truth for grounded signals in the current slice?
6. what does FastAPI own, and what does it explicitly not own?
7. why is `Evaluate(...)` more important than the model's raw `confidence` field?
8. what role does `CompetitionCatalog` play beyond user-facing labels?
9. what is the best cross-layer trace id for linking UI, `.NET`, and FastAPI activity?
10. which external providers are evidence sources in this slice, and which provider is synthesis only?
11. where would a second run type enter the current design?
12. what is the first warning sign that `AgentRunService` needs refactoring?

Say It Out Loud Prompts

1. explain the sports request flow from angular submit to rendered result in under one minute
2. explain why the platform owns provenance and confidence calibration instead of the model
3. explain the difference between collector output, analyzer output, evaluator output, and final composed output
4. explain where trust begins and how that affects persistence and retrieval
5. explain how you would add a new competition without turning the slice chaotic

Before I Change This Flow

- am i changing the request shape across TS, C#, and Python consistently?
- am i preserving `selectedEvent()` as the source of real matchup orientation?
- am i keeping trust and tenant resolution outside the UI and outside FastAPI?
- am i keeping grounded evidence ownership in `.NET`?

- am i preserving evaluator ownership of final confidence?
- am i extending an existing seam instead of adding ad hoc branching in the wrong layer?
- if this adds competition behavior, did i check `CompetitionCatalog` first?
- if this changes a provider contract, did i keep the fallback and `null` behavior honest across `.NET` and FastAPI?